

# Multimedia Computing

By

Minhaz Uddin Ahmed, PhD

Department of Computer Engineering

Inha University Tashkent.

Email: [minhaz.ahmed@gmail.com](mailto:minhaz.ahmed@gmail.com)

# Contents

- ▶ Chapter -10 (gonzalez) and chapter-5 (Richard Szeliski book)
- ▶ Different Image segmentation methods
- ▶ Mean Shift algorithm
- ▶ Spectral clustering
- ▶ Skin tone detection
- ▶ Contour based object detection
- ▶ Segmentation based object detection
- ▶ Face detection
- ▶ Face detection evaluation

# Mean shift algorithm

## Kernel Density Estimation (KDE):

Mean Shift relies on estimating the density of data points in a feature space using a kernel function.

A popular choice is the Gaussian kernel, which gives more weight to nearby points and less weight to distant points.

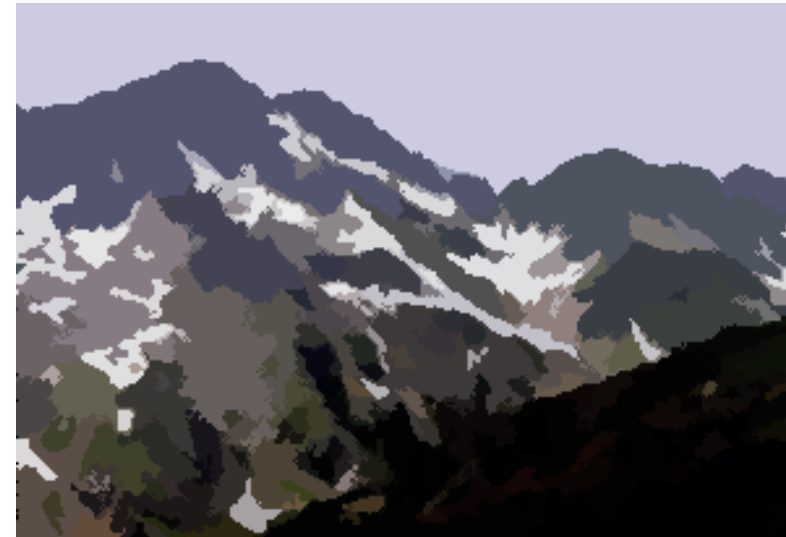
## Bandwidth ( $h$ ):

The bandwidth defines the radius of influence of the kernel function. It controls how far the algorithm looks to determine the density around a point.

## Convergence:

Points are iteratively shifted toward the densest region (the mode of the density function) until they converge, i.e., the shift is negligible.

# Mean shift segmentation results



<http://www.caip.rutgers.edu/~comanici/MSPAMI/msPamiResults.html>

# Mean shift pros and cons

## ▶ Pros

- ▶ Good general-practice segmentation
- ▶ Flexible in number and shape of regions
- ▶ Robust to outliers

## ▶ Cons

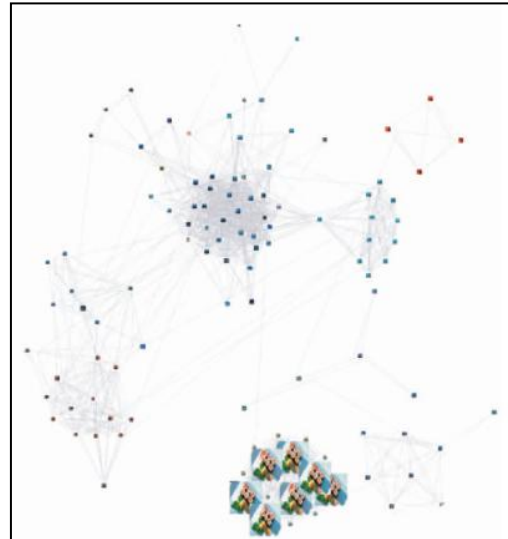
- ▶ Have to choose kernel size in advance
- ▶ Not suitable for high-dimensional features

## ▶ When to use it

- ▶ Over segmentation
- ▶ Multiple segmentations
- ▶ Tracking, clustering, filtering applications

# Spectral clustering

- ▶ **Spectral clustering** is a powerful machine learning algorithm used to group data into clusters based on the relationships (similarity) between points. Instead of working directly in the original data space, it transforms the data into a new space using the **eigenvalues** and **eigenvectors** of a graph that represents the data. It is especially useful for clustering datasets with **non-convex or complex shapes**, where traditional clustering methods like K-Means may fail.



# Spectral clustering

## ► Construct a Similarity Graph:

- Represent the data points as a graph, where:
  - Each data point is a node.
  - The edges between nodes represent the similarity between data points.
- Similarity can be measured using a kernel (e.g., Gaussian kernel)

$$S(i, j) = \exp \left( -\frac{\|x_i - x_j\|^2}{2\sigma^2} \right)$$

If two points are very close, their similarity is high.

# Spectral clustering

**Compute the Laplacian Matrix:**

The Laplacian matrix  $L$  is derived from the similarity matrix  $S$

$$L = D - S$$

Here,

$D$  is the degree matrix (a diagonal matrix where each entry represents the sum of similarities for a node).



# Spectral clustering

## **Compute Eigenvectors:**

Perform eigen decomposition of the Laplacian matrix to obtain its eigenvalues and eigenvectors. Select the eigenvectors corresponding to the smallest  $k$  eigenvalues (other than 0). These eigenvectors form a new space where clustering becomes easier.

## **Cluster in the New Space:**

Treat the rows of the matrix of selected eigenvectors as points in a new  $k$ -dimensional space. Apply a simple clustering algorithm (like K-Means) to group these points.

## **Map the Clusters Back:**

The clusters found in the transformed space correspond to clusters in the original data.

# Practice code spectral clustering

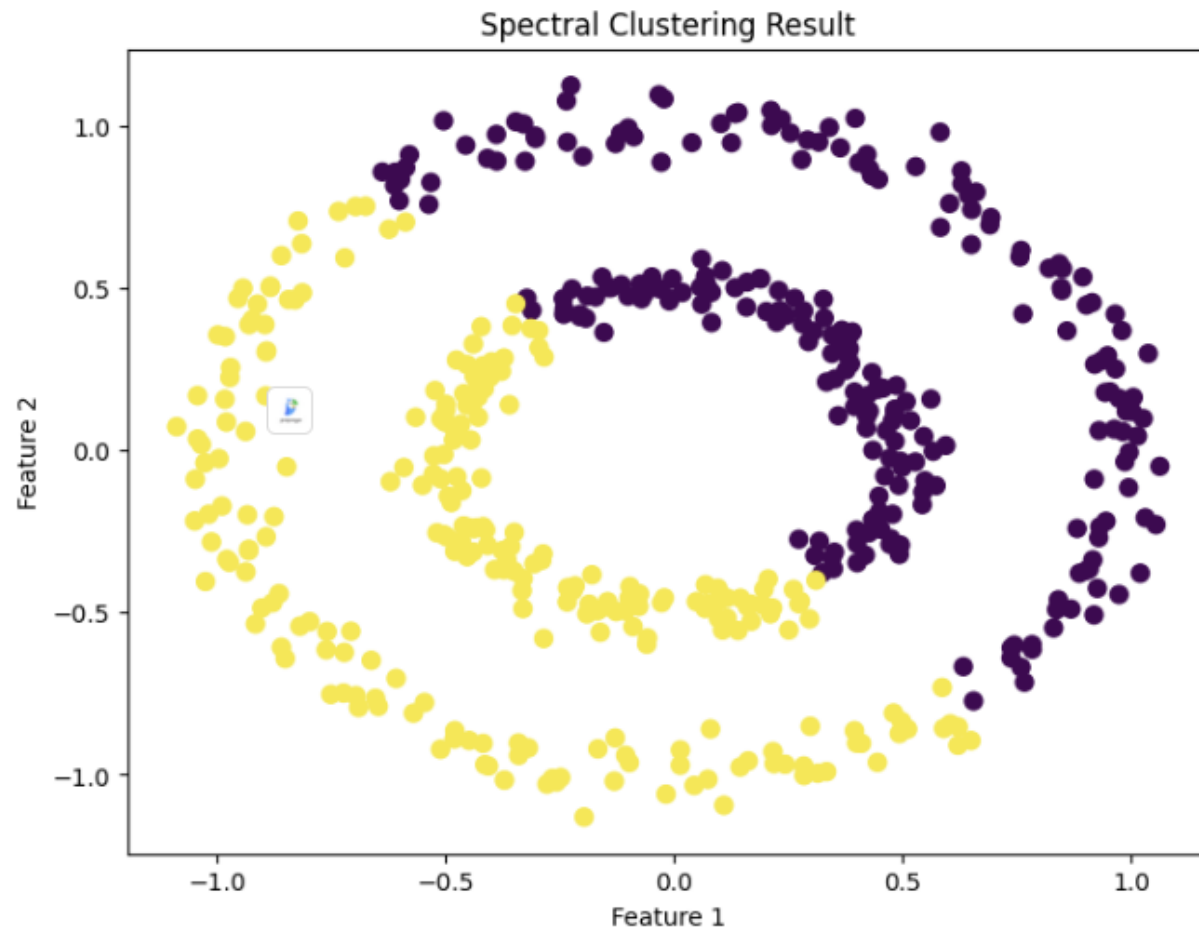
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import SpectralClustering
from sklearn.datasets import make_circles

# Create a dataset of two concentric circles
X, y = make_circles(n_samples=500, factor=0.5, noise=0.05)

# Apply Spectral Clustering
spectral = SpectralClustering(n_clusters=2, affinity='rbf',
gamma=10, random_state=42)
y_pred = spectral.fit_predict(X)

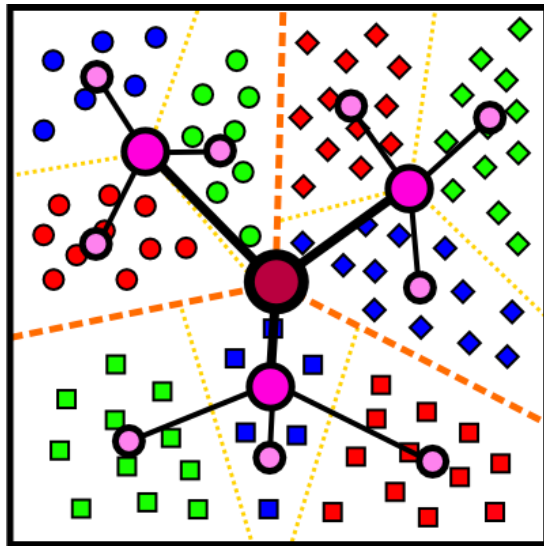
# Plot the results
plt.figure(figsize=(8, 6))
plt.scatter(X[:, 0], X[:, 1], c=y_pred, cmap='viridis',
marker='o', s=50)
plt.title('Spectral Clustering Result')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```

# spectral clustering

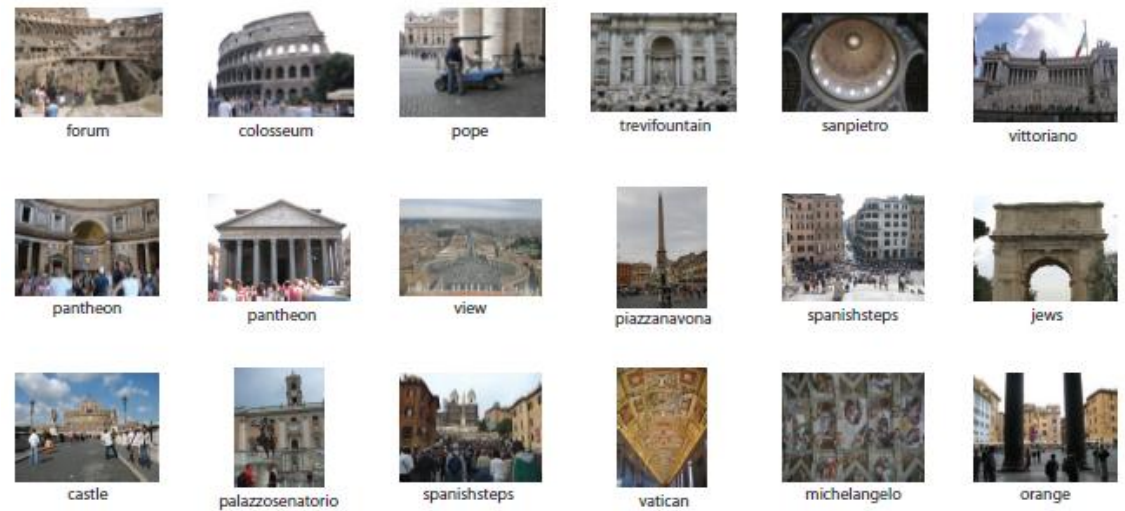


# Summary

- ▶ Summarization: K-means
  - ▶ Aims to preserve variance of original data
  - ▶ Can easily assign new point to a cluster



Quantization for  
computing histograms

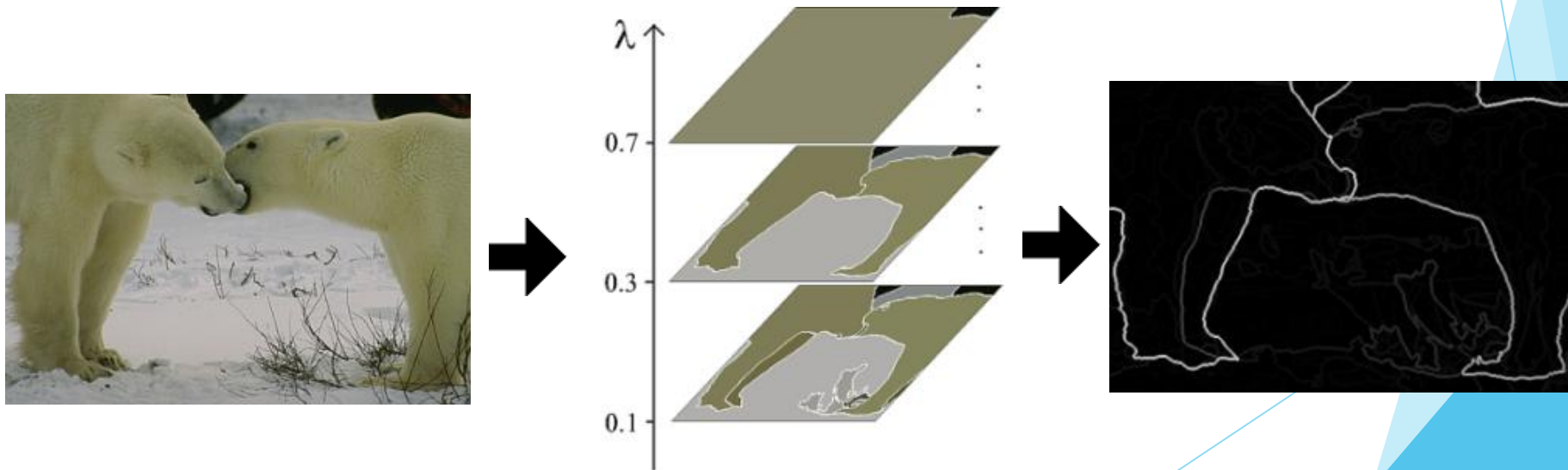


Summary of 20,000 photos of Rome using  
“greedy k-means”

<http://grail.cs.washington.edu/projects/canonview/>

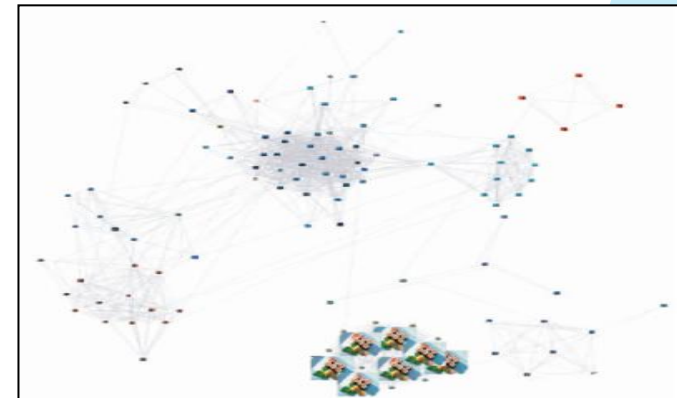
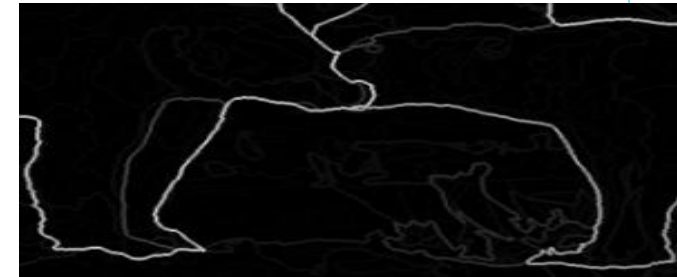
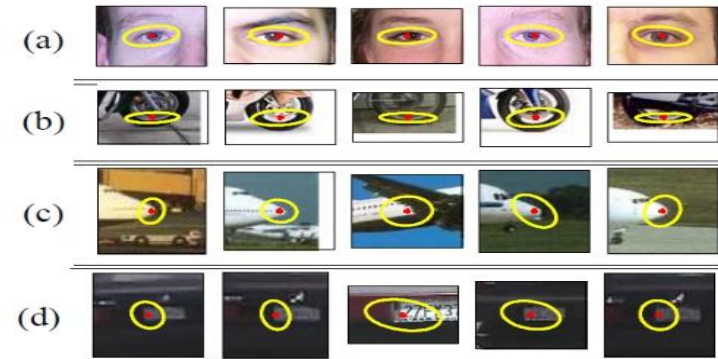
# Summary

- ▶ Image segmentation: agglomerative clustering
  - ▶ More flexible with distance measures (e.g., can be based on boundary prediction)
  - ▶ Adapts better to specific data
  - ▶ Hierarchy can be useful



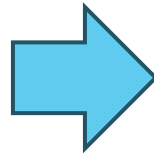
# Summary

- K-means useful for summarization, building dictionaries of patches, general clustering
- Agglomerative clustering useful for segmentation, general clustering
- Spectral clustering useful for determining relevance, summarization, segmentation



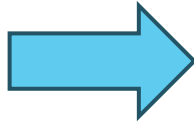


# Skin tone detection



we can see that there are a variety of skin tones to account for. We would find that no single threshold, or adapted threshold would do the job on its own. Therefore, we need to combine multiple together.

# Skin tone detection



Split to HSV



# Skin tone detection

- ▶ With an understanding of the different thresholding types, we can apply the use case of detecting and segmenting of skin tones from an image. This will also cover the use of composite filtering to improve the result if no single threshold will work on its own.

# Skin tone detection code

```
import numpy as np
import cv2

img = cv2.imread('faces.jpeg',1)
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
h = hsv[:, :, 0]
s = hsv[:, :, 1]
v = hsv[:, :, 2]

hsv_split = np.concatenate((h,s,v), axis=1)
cv2.imshow("Split HSV",hsv_split)

ret, min_sat = cv2.threshold(s,40,255, cv2.THRESH_BINARY)
cv2.imshow("Sat Filter",min_sat)

ret, max_hue = cv2.threshold(h,15, 255, cv2.THRESH_BINARY_INV)
cv2.imshow("Hue Filter",max_hue)

final = cv2.bitwise_and(min_sat,max_hue)
cv2.imshow("Final",final)
cv2.imshow("Original",img)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

# Skin tone detection code in colab

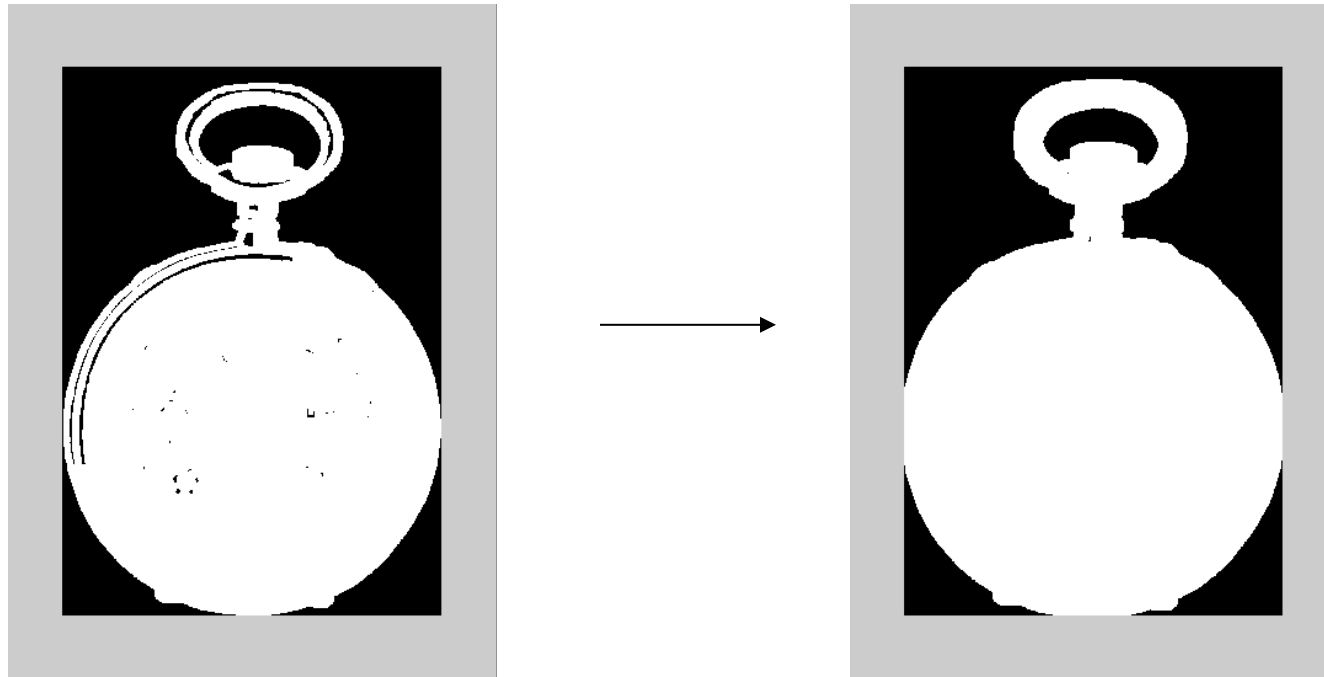
```
import numpy as np
import cv2
from google.colab.patches import cv2_imshow

img = cv2.imread('faces.jpeg',1)
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
h = hsv[:, :, 0]
s = hsv[:, :, 1]
v = hsv[:, :, 2]

hsv_split = np.concatenate((h,s,v), axis=1)
cv2_imshow(hsv_split)
ret, min_sat = cv2.threshold(s, 40, 255, cv2.THRESH_BINARY)
cv2_imshow(min_sat)
ret, max_hue = cv2.threshold(h, 15, 255, cv2.THRESH_BINARY_INV)
cv2_imshow(max_hue)
final = cv2.bitwise_and(min_sat, max_hue)
#cv2_imshow(final)
#cv2_imshow(img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

# Practice contour based object detection

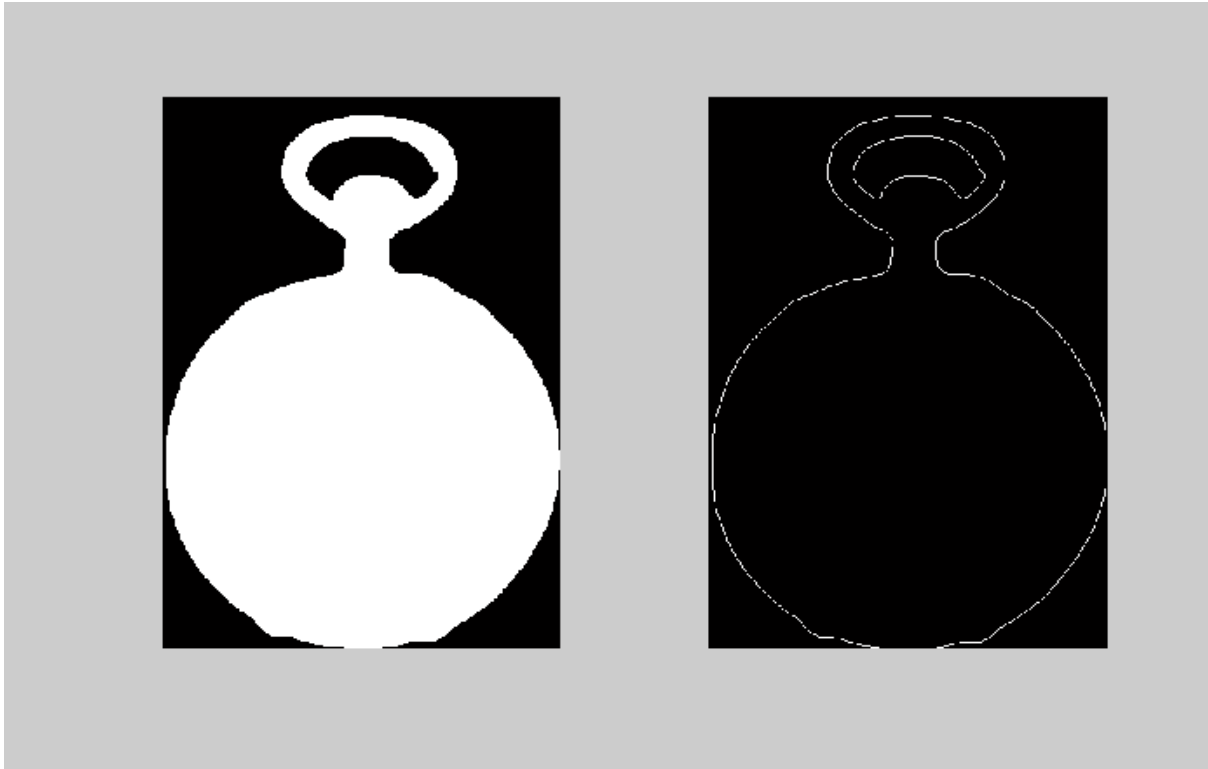
- ▶ Contour based object detection



Erasing the noise using thresholding

# Edge finding

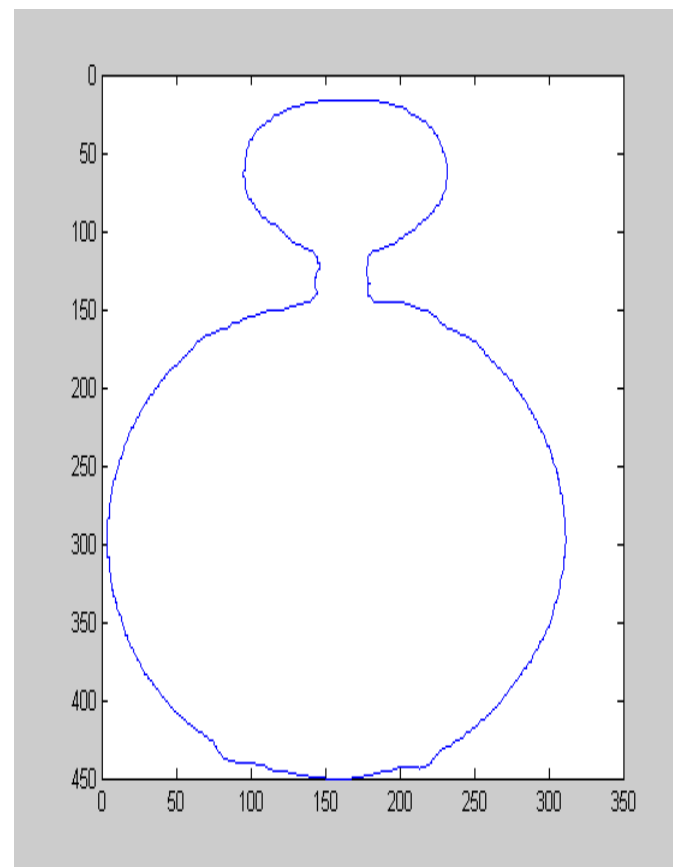
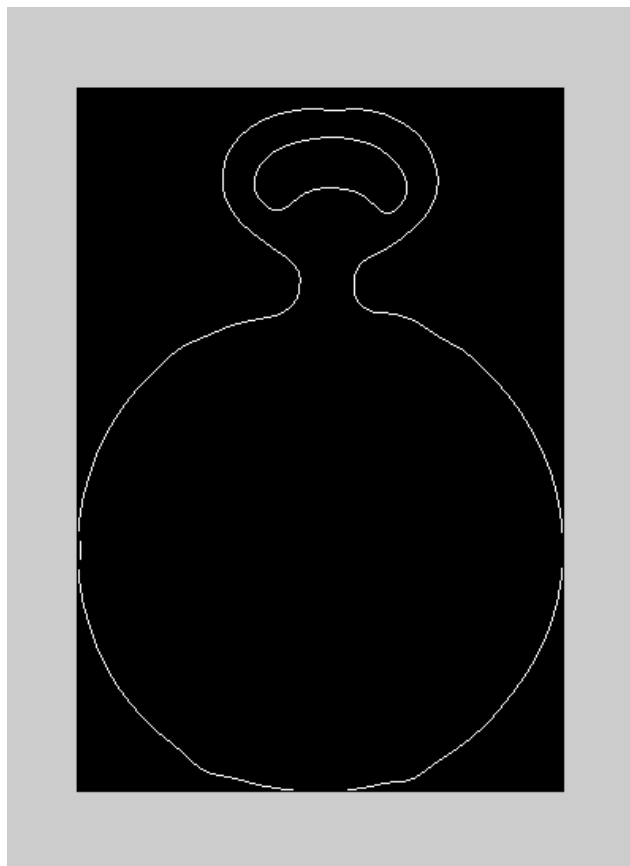
► `img_edge=edge(x);`      matlab



# Contour detection

- Need a starting point;
- Need to decide the direction to travel;
- Need to avoid any circle in traveling.

# Contour detection





# Contours

- ▶ The goal of contours is to take a binary image and create a tightly fitting closed perimeter around all individual objects in the scene. Each perimeter is called a contour.
- ▶ From a mathematical point of view, it is called an iterative energy reduction algorithm, but conceptually, we can think of it as an elastic film that starts on the edges of an image and squeezes in around all the objects and shapes. It creates the boundary around all these objects. One thing to be aware of is the idea of neighborhoods and connectedness.
- ▶ Contours will consider any pixel value above zero as part of the foreground and any other pixels touching or connected to this pixel will be made to be part of the same

# Contours

- ▶ we know contours **will always converge so we'll never be stuck in an infinite loop**. At the end, you have a list of contours and each contour is simply a linear list of points which describe the perimeter of a single object.
- ▶ **Contours is one of the gateways to determining many other useful properties about a single object within an image**, making a very powerful algorithm at our image processing disposal. It moves from the step of object segmentation, often done by thresholding, into the step of object detection.

# Contour object detection

```
import numpy as np
import cv2

img = cv2.imread('detect_blob.png',1)
cv2.imshow("Original", img)
gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
thresh = cv2.adaptiveThreshold(gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, 115, 1)
cv2.imshow("Binary", thresh)

contours, hierarchy = cv2.findContours(thresh, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)

img2 = img.copy()
index = -1
thickness = 4
color = (255, 0, 255)

cv2.drawContours(img2, contours, index, color, thickness)
cv2.imshow("Contours",img2)

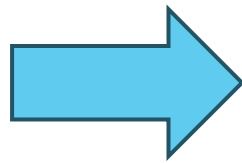
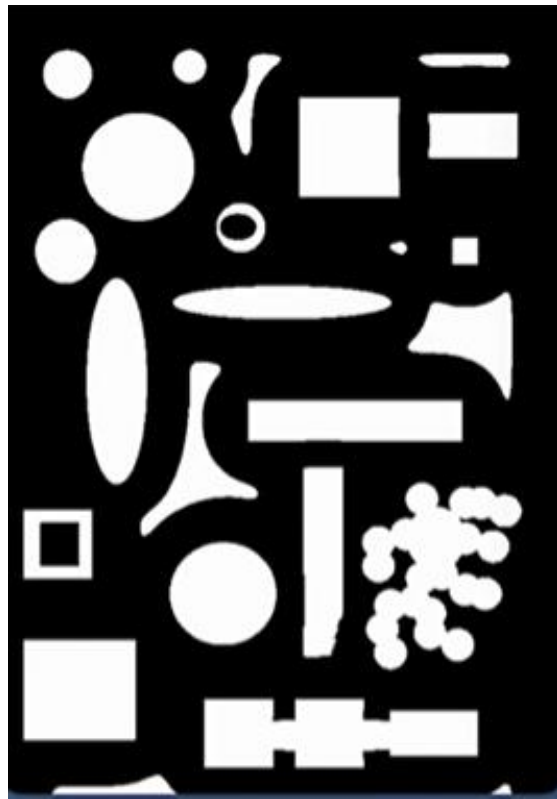
cv2.waitKey(0)
cv2.destroyAllWindows()
```

# Contour object detection colab code

```
import numpy as np
import cv2
from google.colab.patches import cv2_imshow

img = cv2.imread('detect_blob.png',1)
cv2_imshow(img)
gray = cv2.cvtColor(img,cv2.COLOR_RGB2GRAY)
thresh =
cv2.adaptiveThreshold(gray,255,cv2.ADAPTIVE_THRESH_GAUSSIAN_C,cv2.THRESH_BINARY,115,1)
cv2_imshow(thresh)
contours, hierarchy = cv2.findContours(thresh,cv2.RETR_TREE,cv2.CHAIN_APPROX_SIMPLE)
img2 = img.copy()
index = -1
thickness = 4
color = (255,0,255)
cv2.drawContours(img2,contours,index,color,thickness)
cv2_imshow(img2)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

# Contour object detection



# Edge detection

```
import numpy as np
import cv2
#from google.colab.patches import cv2_imshow
img = cv2.imread("tomatoes.jpg",1)
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
res, thresh = cv2.threshold(hsv[:, :, 0], 25, 255, cv2.THRESH_BINARY_INV)
cv2.imshow("Thresh", thresh)

edges = cv2.Canny(img, 100, 200, apertureSize=3)
cv2.imshow("Canny", edges)

edge_inv = 255 - edges

# Use erosion to increase size of the 'border'
kernel = np.ones((3, 3), 'uint8')
erode = cv2.erode(edge_inv, kernel, iterations=1)

canny_thresh = cv2.bitwise_and(erode, thresh)
cv2.imshow("Canny thresh", canny_thresh)
```

# Edge detection

```
contours, hierarchy = cv2.findContours( canny_thresh, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)  
objects = img.copy()
```

```
for c in contours:  
    area = cv2.contourArea(c)  
    if area < 300:  
        # This contour is around something too small for our interest  
        continue  
    print("Area: ", area)  
    cv2.drawContours(objects, [c], -1, (255, 255, 255), 1)  
    M = cv2.moments(c)  
    cx = int(M['m10'] / M['m00'])  
    cy = int(M['m01'] / M['m00'])  
    cv2.circle(objects, (cx, cy), 4, (255, 255, 0), -1)  
cv2.imshow("Final draw-over", objects)  
cv2.waitKey(0)  
cv2.destroyAllWindows()
```

# Ripe object detection

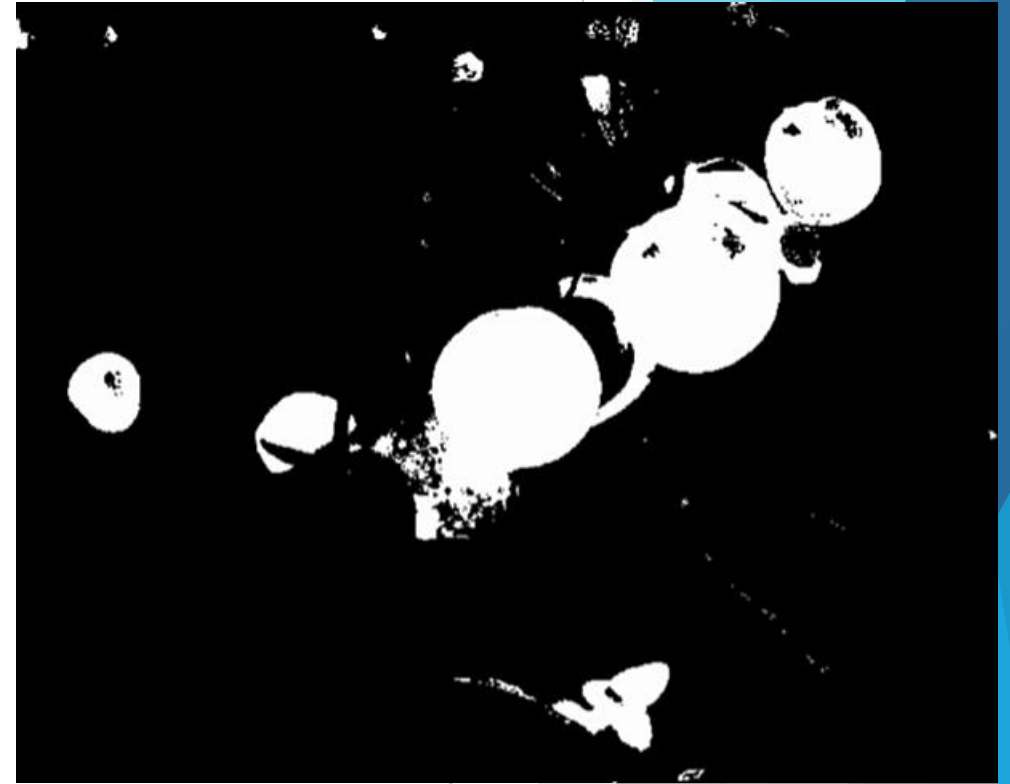




# Step to detect fruit by image processing



Ripe object detection

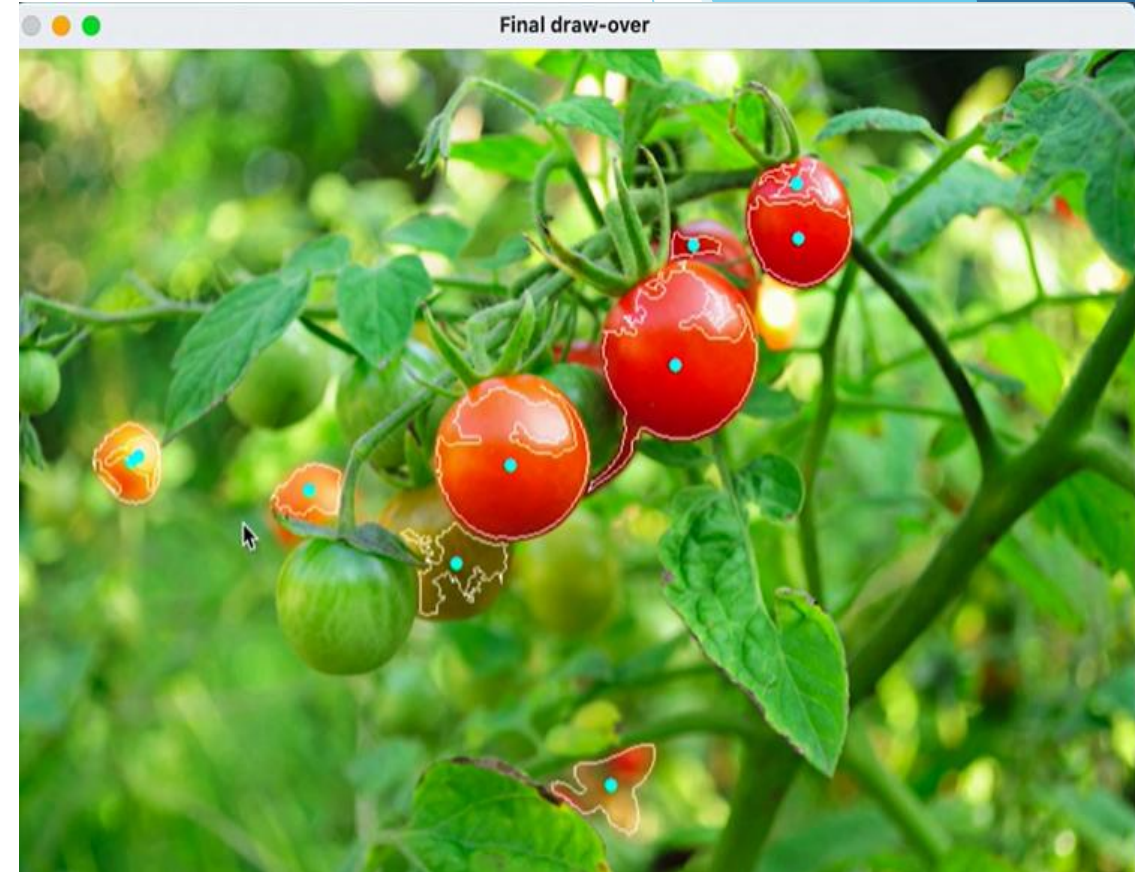
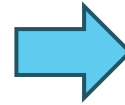


Threshold

# Step to detect fruit by image processing



Canny





# Methods

Simple value-based thresholding

Adaptive thresholding based on local brightness variances

Edge detection to separate object boundaries

Using Gaussian blurs to reduce the impact of image noise

Use of dilation and erosion to reduce speckles or to expand/contract segmented areas



# Segmentation and object detection

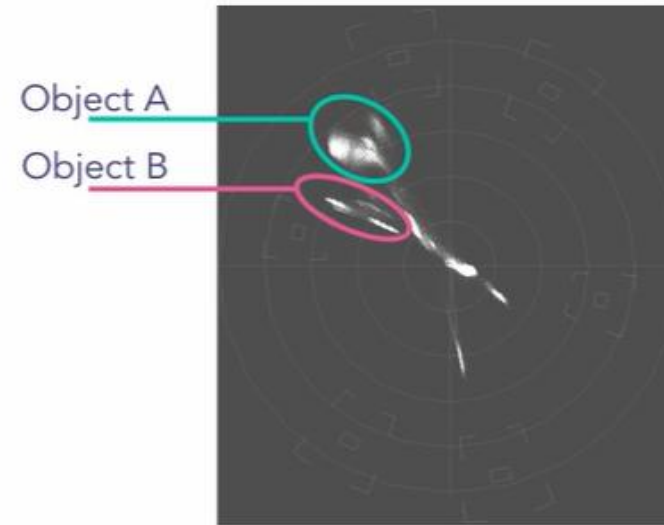
## Some additional techniques

Histogram-based segmentation

Alternate color space transformations

Watershed algorithm

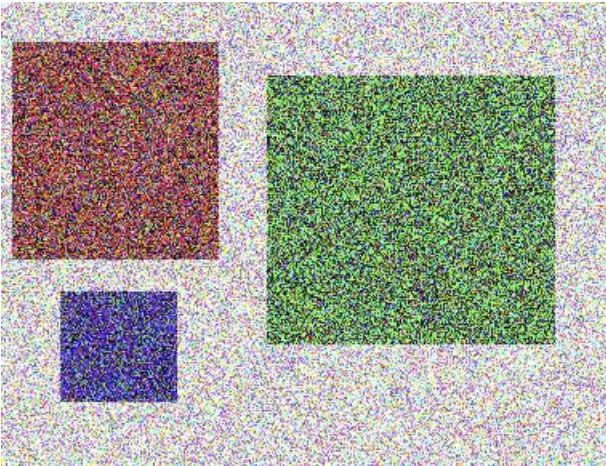
Many more a priori based methods



# Segmentation and object detection

- ▶ You can improve your thresholding by gathering your own global average or mean. [How about object orientation and scale?](#) Can you make assumptions about the size of an object and an image, therefore, allowing you to filter out anything that doesn't fit that size.
- ▶ if you know that the subjects of your image will always be against a black background, it allows you to make different assumptions and processing technique decisions that wouldn't hold up in more general situations. When looking to detect and segment out objects in an image, be aware of all the tools at your disposal.

# Practice exercise



## Identify objects

For all objects in the image, segment them out, draw them on a blank image, and print the perimeter and area

Only draw large objects (area of greater than 1,000 px<sup>2</sup>)

Each object should be drawn with its own color (it does not need to match the source image color)

## Tips

Consider smoothing the image before segmenting

Other imports are allowed, such as `random`

# Practice code Identify objects

```
import numpy as np
import cv2
from random import randint
```

```
img = cv2.imread("fuzzy.png",1)
cv2.imshow("Original", img)
```

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(gray, (5,5), 0)
```

```
thresh = cv2.adaptiveThreshold(
    blur, 255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY_INV, 205, 1)
cv2.imshow("Thresh", thresh)
```

```
contours, hierarchy = cv2.findContours(
    thresh, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)
print(len(contours))
```

# Practice code Identify objects

```
filtered = []
for c in contours:
    if cv2.contourArea(c) < 1000:
        continue
    filtered.append(c)
print(len(filtered))

objects = np.zeros([img.shape[0], img.shape[1], 3], 'uint8')
for c in filtered:
    col = (randint(0,255), randint(0,255), randint(0,255))
    cv2.drawContours(objects, [c], -1, col, -1)
    area = cv2.contourArea(c)
    print(area)

cv2.imshow("Final", objects)

cv2.waitKey(0)
cv2.destroyAllWindows()
```



# Difference between detection and recognition

- ▶ A feature is any part of an image which can be used to understand an image better and to help perform a needed task. Features can be visual components of an image or more mathematical properties of the patterns and arrangement of the pixel values. Good features are invariant to changes like lighting or scaling.

# Object detection

- ▶ Statistical properties of an image or color distribution could also act as features if it fits the problem. Features are identified by classifiers to either detect or recognize objects in an image.
- ▶ Note to be careful how you use the term detection versus recognition. Detection is often a step prior to recognition. For example, with faces, you first might detect whether a face exists in an image and a follow-up step might be to understand that that face matches any image in a database.
- ▶ In this case, features such as the distance between eyes in an image may be used both for the detection process, to see whether or not a face actually exists, but also used as a classifier for the recognition process, to see which face it specifically matches with.

# Object Recognition

Object Detection

Object Identification

Where is Jane?  
Where is a Face?  
Is there a face in the image?



Who is it?  
Is it Jane or Erik?



# General Problems of Recognition

Invariance:

- “External parameters”
  - Pose
  - Illumination
- “Internal parameters”
  - Person identity
  - Facial expression



Applicable to many classes  
of objects

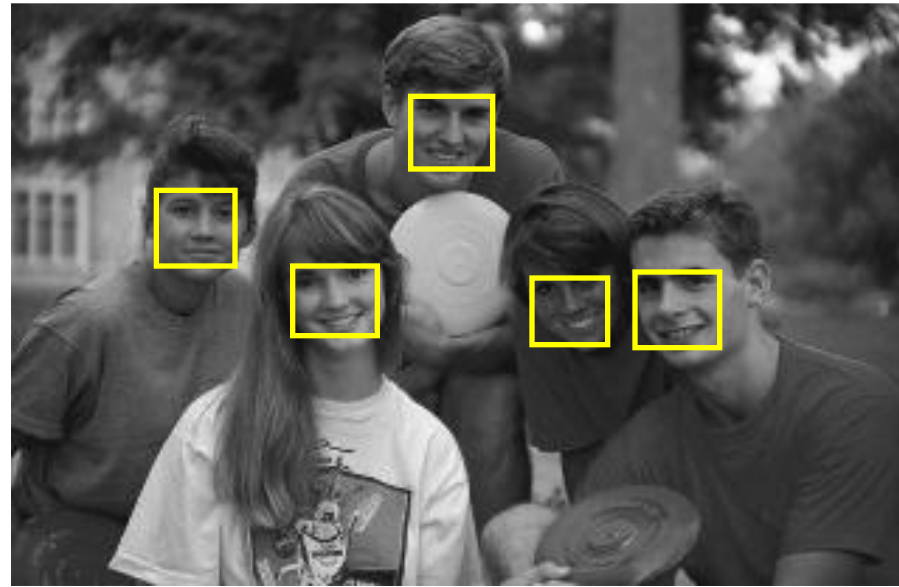


Pattern Recognition for Vision

# Object Detection

## Task

Given an input image, determine if there are objects of a given class (e.g. faces, people, cars..) in the image and where they are located.



Pattern Recognition for Vision

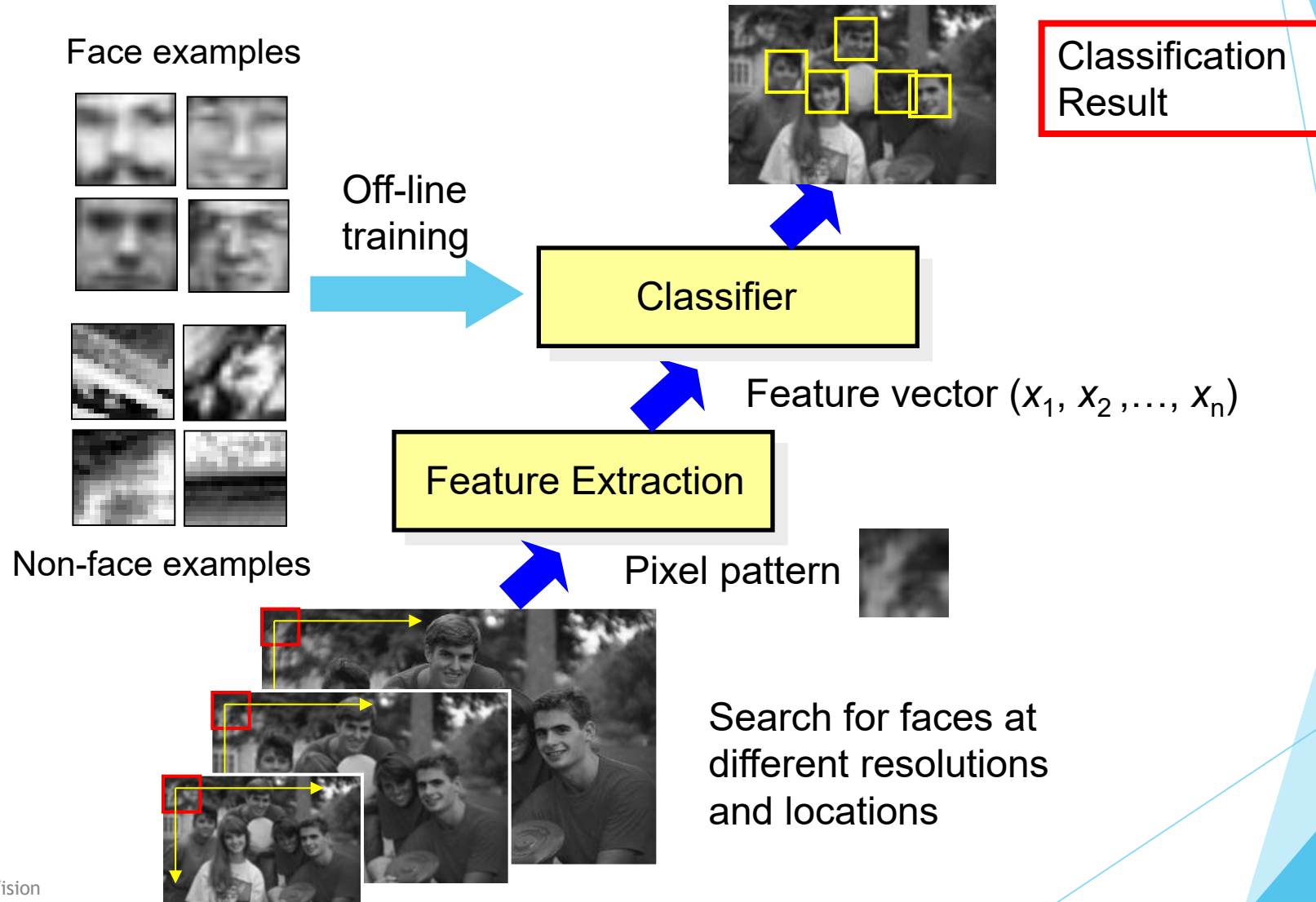
# Detection—Problems

1. Classifier must generalize over all exemplars of one class.
2. Negative class consists of everything else.
3. High accuracy (small FP rate) required for most applications.



Pattern Recognition for Vision

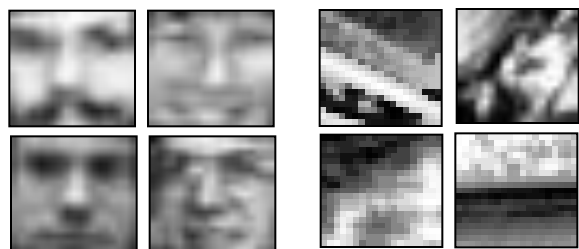
# Face Detection - basic scheme





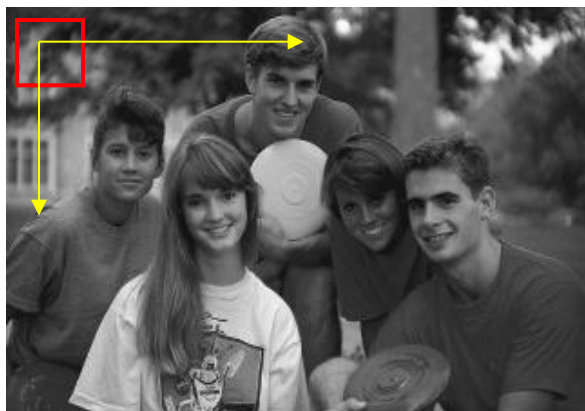
# Training and Testing

## Training Set



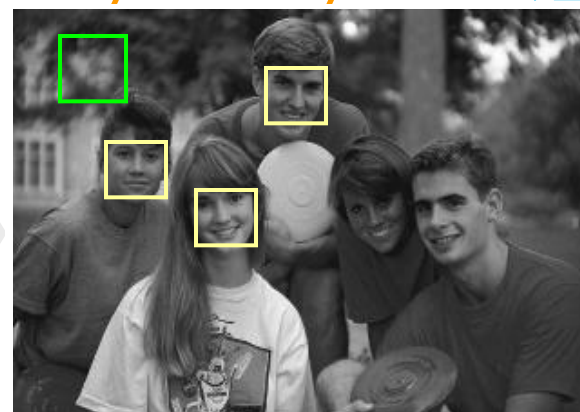
Train Classifier

## Labeled Test Set

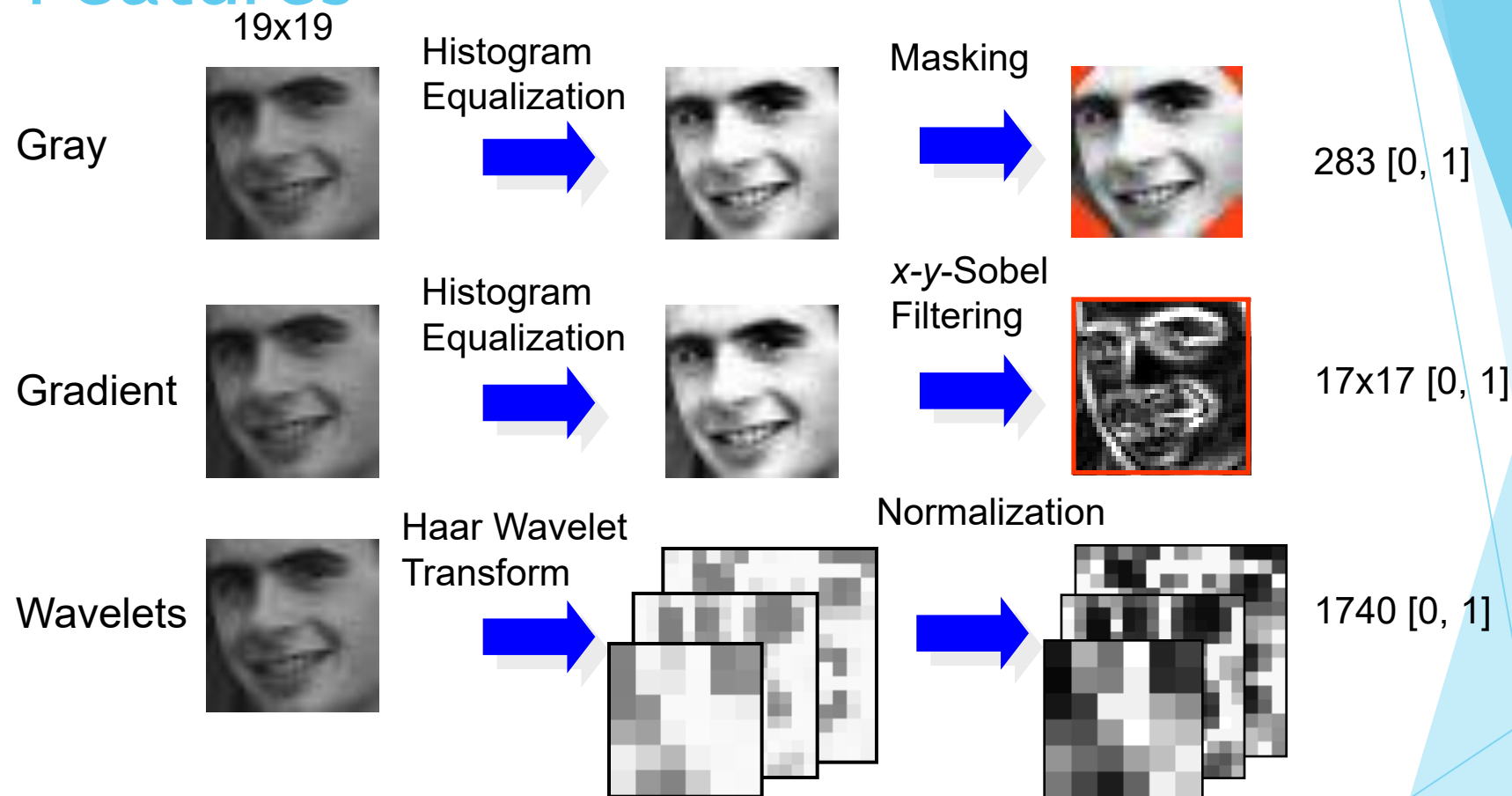


Classify

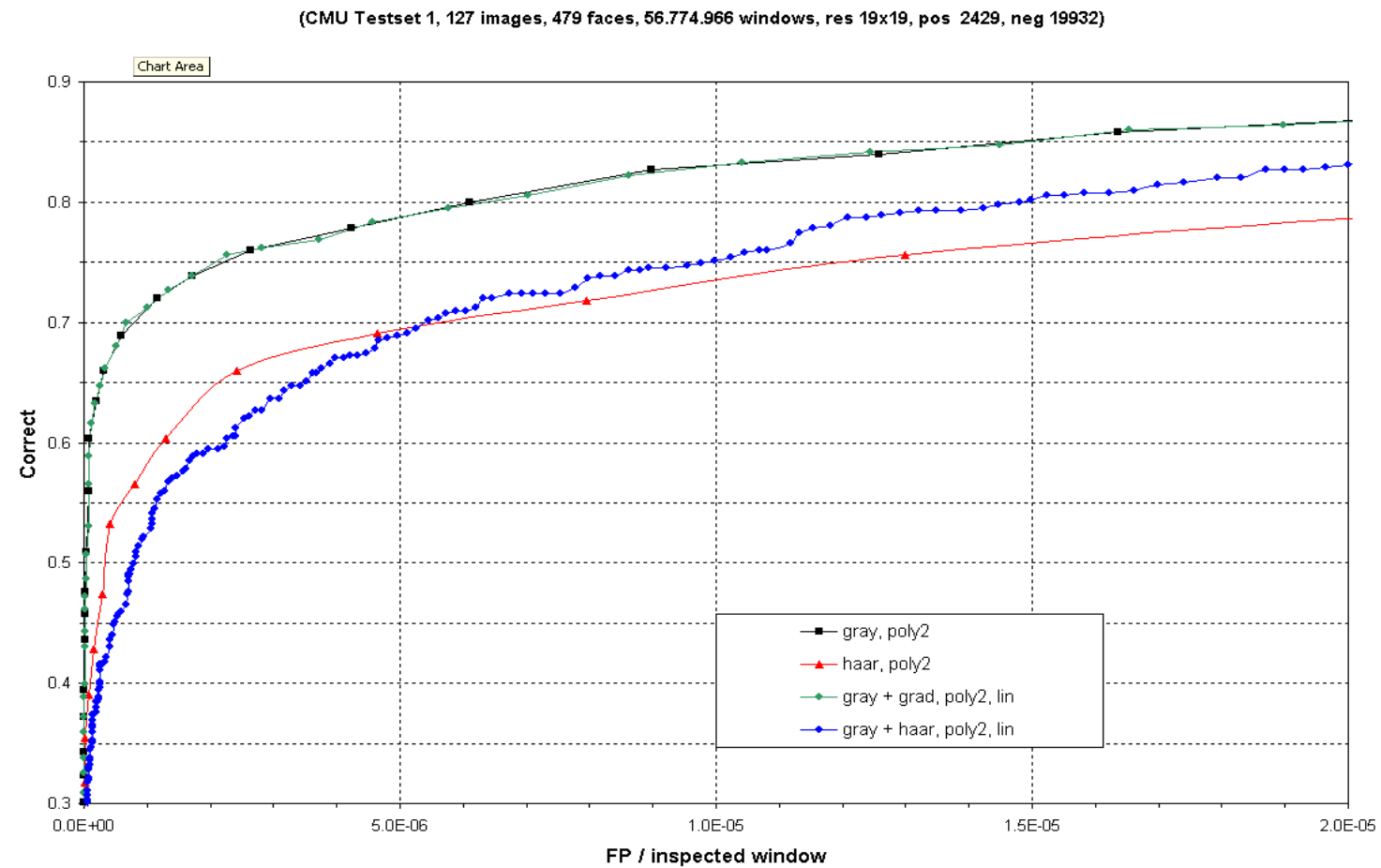
Sensitivity



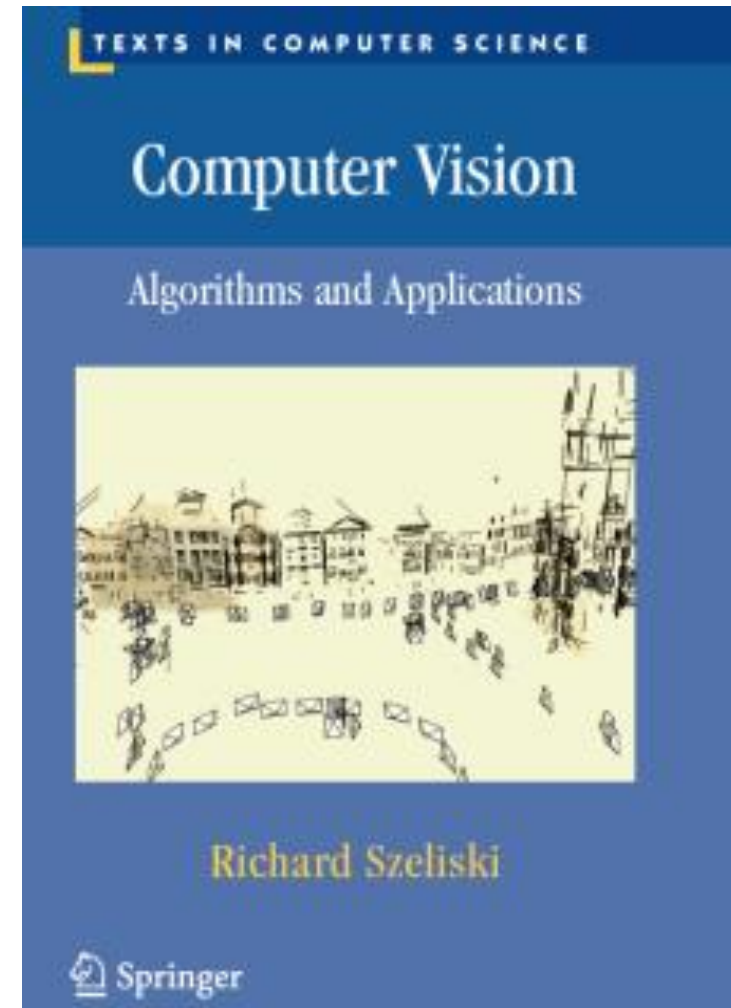
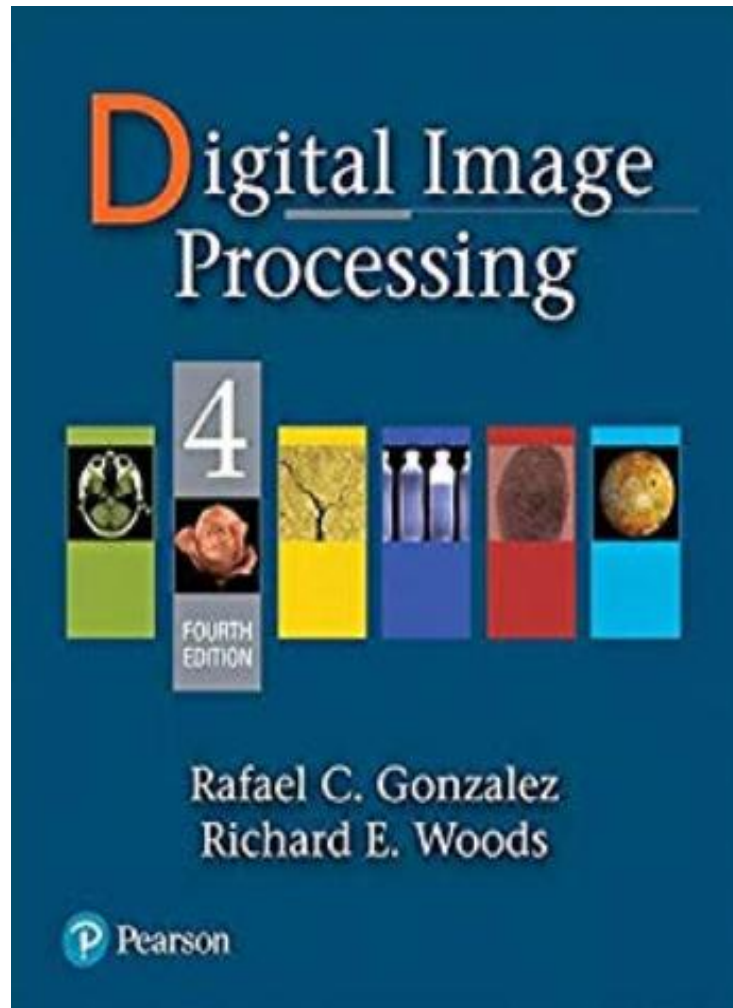
# Image Features



# ROC



# Reference



# Question



Thank you

